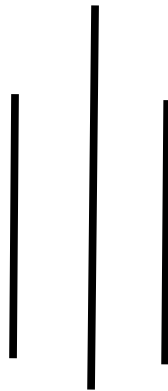


LAB REPORT OF CRYPTOGRAPHY(CSC-316)



Submitted By:

Shudarsan Paudel
BscCSIT 5th Sem

Submitted To:

Deep Raj Bhujel

Signature

TABLE OF CONTENTS

SN.	Topic	Date	Signature
1	Cesar Cipher	2082-12-07	
2	Monoalphabetic Cipher	2082-12-07	
3	Playfair Cipher	2082-12-08	
4	Hill Cipher	2082-12-08	
5	Vigenere Cipher	2082-12-08	
6	OTP (One-Time Pad) Cipher	2082-12-09	
7	Rail-Fence Cipher	2082-12-09	
8	Euclidean and Extended Euclidean Algorithm	2082-12-10	
9	Diffie-Hellman Keyexchange	2082-12-10	
10	Rivest Shamir and Adleman (RSA) Algorithm	2082-12-10	

LAB 1: Implementing Ceasar Cipher

Objective

To encrypt and decrypt the given word

Tools Used

VS-Code, gcc compiler

Algorithm: Ceasar Cipher

1. Start
2. Declare variables (m, k, c) for message, key and counter respectively
3. Take message as an input from the user
4. Take key (shift value) from the user
5. Encrypt the given message:
for each character, if it is a letter, shift it forward by the key else leave it unchanged
6. Display the encrypted message
7. Decrypt message:
for each character, if it is a letter, shift it backward by the key else leave it unchanged
8. Display the decrypted message
9. End

Source Code of Ceasar Cipher

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void encrypt(char text[], int key) {
    for (int i = 0; text[i] != '\0'; i++) {
        if (isupper(text[i])) {
            text[i] = (text[i] - 'A' + key) % 26 + 'A';
        }
        else if (islower(text[i])) {
            text[i] = (text[i] - 'a' + key) % 26 + 'a';
        }
    }
}
```

4

```
}  
  
void decrypt(char text[], int key) {  
    for (int i = 0; text[i] != '\0'; i++) {  
        if (isupper(text[i])) {  
            text[i] = (text[i] - 'A' - key + 26) % 26 + 'A';  
        }  
        else if (islower(text[i])) {  
            text[i] = (text[i] - 'a' - key + 26) % 26 + 'a';  
        }  
    }  
}  
  
int main() {  
    char text[100];  
    int key;  
  
    printf("Enter a word: ");  
    scanf("%s", text);  
  
    printf("Enter key (shift): ");  
    scanf("%d", &key);  
  
    encrypt(text, key);  
    printf("Encrypted text: %s\n", text);  
  
    decrypt(text, key);  
    printf("Decrypted text: %s\n", text);  
  
    return 0;  
}
```

Output:

```
→ cryptography git:(main) × ./c  
Enter a word: information  
Enter key (shift): 3  
Encrypted text: lqirupdwlrq  
Decrypted text: information
```

Conclusion:

The Caesar Cipher is a simple and easy-to-implement encryption technique that shifts letters by a fixed value, but it is not very secure due to its limited key space.

LAB 2: Implementing Monoalphabetic Cipher

Objective

To encrypt and decrypt the given word

Tools Used

VS-Code, gcc compiler

Algorithm: Substitution Cipher using Key Phrase

1) Start

2) Declare Variables:

- text[100] → stores the plaintext message
- keymap[26] → stores the substitution mapping
- key_phrase[50] → stores the key phrase
- encrypted[100] → stores the encrypted message
- used[26] → tracks which letters have been used
- inv_keymap[26] → stores the inverse mapping for decryption

3) Input:

- Read plaintext message from the user
- Read key phrase from the user

4) Generate Keymap:

- Initialize used[] array to track used letters
- For each character in the key phrase:
 - If it is a unique letter:
 - Add it to keymap in order
 - Mark it as used
- Fill remaining positions of keymap with unused letters from a to z

5) Display Keymap:

- Print the substitution mapping:
a - z $\rightarrow$ keymap[0 - 25]

6) Encrypt the Message:

- Copy plaintext into encrypted
- For each character in encrypted:
 - If uppercase:
 - Convert to lowercase, substitute using keymap, convert back to uppercase
 - If lowercase:
 - Substitute using keymap
 - If non-letter:
 - Leave unchanged

7) Display Encrypted Message

8) Decrypt the Message:

- Create inv_keymap (reverse mapping of keymap)
- For each character in encrypted:
 - If uppercase:
 - Convert to lowercase, find original using inv_keymap, convert back to uppercase
 - If lowercase:
 - Find original using inv_keymap
 - If non-letter:
 - Leave unchanged

9) Display Decrypted Message

10) End

Source Code of Monoalphabetic Cipher

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void generate_keymap(char *key_phrase, char keymap[26]) {
// Fixed: char *key_phrase
```

```

int used[26] = {0};
int idx = 0;

// Place key phrase letters first (unique, lowercase)
for (int i = 0; key_phrase[i] != '\0' && idx < 26; i++) {
    int pos = tolower(key_phrase[i]) - 'a';
    if (pos >= 0 && pos < 26 && !used[pos]) {
        keymap[idx++] = 'a' + pos;
        used[pos] = 1;
    }
}

// Fill remaining with unused letters
for (int i = 0; i < 26 && idx < 26; i++) {
    if (!used[i]) {
        keymap[idx++] = 'a' + i;
    }
}

void encrypt(char text[], char keymap[26]) {
    for (int i = 0; text[i] != '\0'; i++) {
        if (isupper(text[i])) {
            text[i] = keymap[text[i] - 'A'] - 'a' + 'A';
        }
        else if (islower(text[i])) {
            text[i] = keymap[text[i] - 'a'];
        }
    }
}

void decrypt(char text[], char keymap[26]) {
    // Create inverse mapping for decryption
    char inv_keymap[26];
    for (int i = 0; i < 26; i++) {
        inv_keymap[keymap[i] - 'a'] = i + 'a';
    }

    for (int i = 0; text[i] != '\0'; i++) {
        if (isupper(text[i])) {
            text[i] = inv_keymap[text[i] - 'A'] - 'a' + 'A';
        }
        else if (islower(text[i])) {
            text[i] = inv_keymap[text[i] - 'a'];
        }
    }
}

int main() {
    char text[100];
    char keymap[26];
    char key_phrase[50]; // Fixed: proper name

```

```

printf("Enter a word: ");
scanf("%s", text);

printf("Enter key phrase: ");
scanf("%s", key_phrase);

generate_keymap(key_phrase, keymap); // Fixed: passes char*
// Show the key mapping
printf("Key mapping: abcdefghijklmnopqrstuvwxyz -> ");
for (int i = 0; i < 26; i++) {
    printf("%c", keymap[i]);
}
printf("\n");

// Make copy for encryption demo
char encrypted[100];
strcpy(encrypted, text);
encrypt(encrypted, keymap);
printf("Encrypted text: %s\n", encrypted);

// Decrypt back to original
decrypt(encrypted, keymap);
printf("Decrypted text: %s\n", encrypted);

return 0;
}

```

Output:

```

→ cryptography git:(main) × ./g
Enter a word: department
Enter key phrase: shawn
Key mapping: abcdefghijklmnopqrstuvwxyz -> shawnbcdefgijklmopqrtuvwxyz
Encrypted text: wnmsprjnkr
Decrypted text: department

```

Conclusion:

The Monoalphabetic Cipher improves security by using a random substitution of letters, making it harder to break than Caesar Cipher, but it is still vulnerable to frequency analysis.

LAB 3: Implementing Playfair Cipher

Objective

To encrypt and decrypt the given word

Tools Used

VS-Code, gcc compiler

Algorithm: Playfair Cipher (Encryption and Decryption using Key Square)

1) Start

2) Declare Variables:

- `text[200]` → stores input plaintext
 - `encrypted[200]` → stores encrypted text
 - `key_phrase[50]` → stores key phrase
 - `key_square[5][5]` → stores 5×5 Playfair matrix
 - `grid[25]` → temporary array for key generation
 - `used[26]` → tracks used letters
 - `row1, col1, row2, col2` → positions of characters in key square
-

3) Input:

- Read text from user
 - Read key phrase from user
-

4) Generate Key Square:

- Initialize `used[26] = 0`
 - Treat **I and J as the same letter (J → I)**
 - For each character in key phrase:
 - Convert to lowercase
 - If it is a letter and not already used:
 - Add to `grid[]`
 - Mark as used
 - Fill remaining `grid[]` with unused letters (excluding J)
 - Fill `key_square[5][5]` row-wise using `grid[]`
-

5) Display Key Square:

- Print the 5×5 matrix

6) Prepare Text:

- Remove non-alphabet characters
 - Convert all letters to uppercase
 - Replace J with I
 - Split repeated letters in a pair by inserting 'X'
 - If length is odd, append 'X' at the end
-

7) Encryption Process:

- Divide text into pairs of two letters
- For each pair:
 - Find positions (row1, col1) and (row2, col2) in key square

Case 1: Same Row

- Replace each letter with the letter to its right (circular shift)

Case 2: Same Column

- Replace each letter with the letter below it (circular shift)

Case 3: Rectangle Rule

- Replace each letter with the letter in same row but opposite column
 - Store result in encrypted
-

8) Display Encrypted Text

9) Decryption Process:

- Divide encrypted text into pairs
- For each pair:
 - Find positions (row1, col1) and (row2, col2)

Case 1: Same Row

- Replace each letter with the letter to its left (circular shift)

Case 2: Same Column

- Replace each letter with the letter above it (circular shift)

Case 3: Rectangle Rule

- Replace each letter with the letter in same row but opposite column
- Remove padding 'X' if added at end

10) Display Decrypted Text

11) End

Source Code of Playfair Cipher

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void generate_key_square(char *key_phrase, char key_square[5][5]) {
    char grid[25];
    int used[26] = {0};
    int idx = 0;

    // Add key phrase letters (I/J treated as same)
    for (int i = 0; key_phrase[i] != '\0' && idx < 25; i++) {
        char c = tolower(key_phrase[i]);
        if (c == 'j') c = 'i'; // Playfair rule: I/J same
        int pos = c - 'a';
        if (pos >= 0 && pos < 26 && pos != 9 && !used[pos]) { // Skip J
            grid[idx++] = 'a' + pos;
            used[pos] = 1;
        }
    }

    // Fill with remaining alphabet (skip J)
    for (int i = 0; i < 26 && idx < 25; i++) {
        if (i != 9 && !used[i]) { // Skip J
            grid[idx++] = 'a' + i;
        }
    }
}
```

```

        used[i] = 1;
    }
}

// Fill 5x5 grid
int k = 0;
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        key_square[i][j] = grid[k++];
    }
}

void find_position(char c, char key_square[5][5], int *row, int *col) {
    if (tolower(c) == 'j') c = 'i';
    char lc = tolower(c);

    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            if (key_square[i][j] == lc) {
                *row = i;
                *col = j;
                return;
            }
        }
    }
}

void prepare_text(char *text) {
    // Convert to uppercase, remove non-letters, add X padding, split double letters
    char result[200] = {0};
    int j = 0;

    for (int i = 0; text[i] != '\0'; i++) {
        if (isalpha(text[i])) {

```

```
        result[j++] = toupper(text[i]);
    }
}
```

```
// Split double letters and pad with X
```

```
for (int i = 0; i < j; i++) {
    if (i < j - 1 && result[i] == result[i+1]) {
        result[i+1] = 'X';
        i++; // Skip next char
    }
}
```

```
// Pad to even length
```

```
if (j % 2 != 0) {
    result[j++] = 'X';
}
```

```
strcpy(text, result);
```

```
}
```

```
void encrypt(char text[], char key_square[5][5]) {
```

```
    prepare_text(text);
```

```
    for (int i = 0; i < strlen(text); i += 2) {
```

```
        int row1, col1, row2, col2;
```

```
        find_position(text[i], key_square, &row1, &col1);
```

```
        find_position(text[i+1], key_square, &row2, &col2);
```

```
        if (row1 == row2) { // Same row: shift right
```

```
            text[i] = key_square[row1][(col1 + 1) % 5];
```

```
            text[i+1] = key_square[row2][(col2 + 1) % 5];
```

```
        }
```

```
        else if (col1 == col2) { // Same column: shift down
```

```
            text[i] = key_square[(row1 + 1) % 5][col1];
```

```
            text[i+1] = key_square[(row2 + 1) % 5][col2];
```

```

    }
    else { // Rectangle: swap columns
        text[i] = key_square[row1][col2];
        text[i+1] = key_square[row2][col1];
    }
}
}

void decrypt(char text[], char key_square[5][5]) {
    for (int i = 0; i < strlen(text); i += 2) {
        int row1, col1, row2, col2;
        find_position(text[i], key_square, &row1, &col1);
        find_position(text[i+1], key_square, &row2, &col2);

        if (row1 == row2) { // Same row: shift left
            text[i] = key_square[row1][(col1 + 4) % 5];
            text[i+1] = key_square[row2][(col2 + 4) % 5];
        }
        else if (col1 == col2) { // Same column: shift up
            text[i] = key_square[(row1 + 4) % 5][col1];
            text[i+1] = key_square[(row2 + 4) % 5][col2];
        }
        else { // Rectangle: swap columns
            text[i] = key_square[row1][col2];
            text[i+1] = key_square[row2][col1];
        }
    }

    // Remove padding X's (simple version)
    for (int i = strlen(text) - 1; i > 0; i--) {
        if (text[i] == 'X' && (i == strlen(text) - 1 || !isalpha(text[i-1]))) {
            text[i] = '\0';
        } else {
            break;
        }
    }
}

```

```
}
```

```
void print_key_square(char key_square[5][5]) {  
    printf("Key Square:\n");  
    for (int i = 0; i < 5; i++) {  
        for (int j = 0; j < 5; j++) {  
            printf("%c ", key_square[i][j]);  
        }  
        printf("\n");  
    }  
    printf("\n");  
}
```

```
int main() {  
    char text[200];  
    char key_square[5][5];  
    char key_phrase[50];  
  
    printf("Enter text: ");  
    fgets(text, sizeof(text), stdin);  
    text[strcspn(text, "\n")] = 0; // Remove newline  
  
    printf("Enter key phrase: ");  
    scanf("%s", key_phrase);  
  
    generate_key_square(key_phrase, key_square);  
    print_key_square(key_square);  
  
    // Encrypt  
    char encrypted[200];  
    strcpy(encrypted, text);  
    encrypt(encrypted, key_square);  
    printf("Encrypted: %s\n", encrypted);  
  
    // Decrypt
```

```

    decrypt(encrypted, key_square);

    printf("Decrypted: %s\n", encrypted);

    return 0;
}

```

Output:

```

→ cryptography git:(main) × ./playfair
Enter text: development
Enter key phrase: function
Key Square:
f u n c t
i o a b d
e g h k l
m p q r s
v w x y z

Encrypted: ilfmgdqphfnz
Decrypted: developmentx

```

LAB 4: Implementing Hill Cipher

Objective

To encrypt and decrypt the given word

Tools Used

VS-Code, gcc compiler\

Algorithm: Hill Cipher

1) Start

2) Input:

- Read plaintext message
- Read key (9 characters)

3) Generate Key Matrix:

- Convert key into 3×3 matrix using A=0, B=1, ... Z=25
- Fill matrix row-wise

4) Prepare Message:

- Convert message to uppercase

- Remove non-letters
- Divide into blocks of 3 characters
- Pad with 'A' if needed

5) Encryption:

- For each block:
 - Convert letters to numbers (A=0 to Z=25)
 - Multiply with key matrix
 - Take mod 26
 - Convert result back to letters
- Store encrypted text

6) Display Encrypted Text

7) Decryption:

- Find inverse of key matrix (mod 26)
- For each encrypted block:
 - Convert to numbers
 - Multiply with inverse matrix
 - Take mod 26
 - Convert back to letters

8) Display Decrypted Text

9) End

Source code of Hill Cipher

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

#define SIZE 3

#define MOD 26

void getKeyMatrix(char *key, int keyMatrix[SIZE][SIZE]) {
    int k = 0, i, j;
    for (i = 0; i < SIZE; i++) {
```

```

    for (j = 0; j < SIZE; j++) {
        while (!isalpha(key[k])) k++;
        keyMatrix[i][j] = toupper(key[k++]) - 'A';
    }
}

}

void matrixMultiply(int mat1[SIZE][SIZE], int mat2[SIZE][SIZE], int result[SIZE][SIZE]) {
    int i, j, k;
    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            result[i][j] = 0;
            for (k = 0; k < SIZE; k++) {
                result[i][j] = (result[i][j] + mat1[i][k] * mat2[k][j]) % MOD;
            }
        }
    }
}

void encrypt(char message[], int keyMatrix[SIZE][SIZE], char cipher[]) {
    int msgVector[SIZE][1], cipherVector[SIZE][1];
    int i, len = strlen(message);

    strcpy(cipher, message);

    for (i = 0; i < len; i += SIZE) {
        int j;
        for (j = 0; j < SIZE; j++) {
            if (i + j < len) {
                msgVector[j][0] = toupper(cipher[i+j]) - 'A';
            } else {
                msgVector[j][0] = 0; // pad with A
            }
        }
    }
}

```

```

    int x;
    for (x = 0; x < SIZE; x++) {
        cipherVector[x][0] = 0;
        int y;
        for (y = 0; y < SIZE; y++) {
            cipherVector[x][0] = (cipherVector[x][0] + keyMatrix[x][y] *
                                msgVector[y][0]) % MOD;
        }
    }

    for (j = 0; j < SIZE; j++) {
        cipher[i+j] = cipherVector[j][0] + 'A';
    }
}
cipher[len] = '\0';
}

void decrypt(char cipher[], int keyMatrix[SIZE][SIZE], char message[]) {
    int invMatrix[SIZE][SIZE];

    // Simple 3x3 inverse calculation (determinant must be coprime with 26)
    int det = (keyMatrix[0][0]*(keyMatrix[1][1]*keyMatrix[2][2] - keyMatrix[1][2]*keyMatrix[2][1]) -
              keyMatrix[0][1]*(keyMatrix[1][0]*keyMatrix[2][2] - keyMatrix[1][2]*keyMatrix[2][0]) +
              keyMatrix[0][2]*(keyMatrix[1][0]*keyMatrix[2][1] - keyMatrix[1][1]*keyMatrix[2][0]));

    int detInv = 0;
    for (int i = 1; i < MOD; i++) {
        if ((det * i) % MOD == 1) {
            detInv = i;
            break;
        }
    }

    // Adjugate matrix (simplified for demo)

```

```

    invMatrix[0][0] = ((keyMatrix[1][1]*keyMatrix[2][2] - keyMatrix[1][2]*keyMatrix[2][1]) * detInv) %
MOD;

    invMatrix[0][1] = ((keyMatrix[0][2]*keyMatrix[2][1] - keyMatrix[0][1]*keyMatrix[2][2]) * detInv) %
MOD;

    invMatrix[1][0] = ((keyMatrix[1][2]*keyMatrix[2][0] - keyMatrix[1][0]*keyMatrix[2][2]) * detInv) %
MOD;

    invMatrix[1][1] = ((keyMatrix[0][0]*keyMatrix[2][2] - keyMatrix[0][2]*keyMatrix[2][0]) * detInv) %
MOD;

    int len = strlen(cipher);

    strcpy(message, cipher);

    for (int i = 0; i < len; i += SIZE) {
        int msgVector[SIZE][1], plainVector[SIZE][1];

        int j;
        for (j = 0; j < SIZE; j++) {
            msgVector[j][0] = toupper(message[i+j]) - 'A';
        }

        int x;
        for (x = 0; x < SIZE; x++) {
            plainVector[x][0] = 0;

            int y;
            for (y = 0; y < SIZE; y++) {
                plainVector[x][0] = (plainVector[x][0] + invMatrix[x][y] *
msgVector[y][0]) % MOD;
            }
        }

        for (j = 0; j < SIZE; j++) {
            message[i+j] = plainVector[j][0] + 'A';
        }
    }

    void printMatrix(int matrix[SIZE][SIZE]) {
        int i, j;

```

```
for (i = 0; i < SIZE; i++) {
    for (j = 0; j < SIZE; j++) {
        printf("%2d ", matrix[i][j]);
    }
    printf("\n");
}
printf("\n");
}

int main() {
    char text[100], key[20] = "GYBNQKURP", encrypted[100], decrypted[100];
    int keyMatrix[SIZE][SIZE];

    printf("Enter plaintext: ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, "\n")] = 0;

    printf("Key (9 chars, default GYBNQKURP): ");
    scanf("%s", key);

    getKeyMatrix(key, keyMatrix);
    printf("Key Matrix:\n");
    printMatrix(keyMatrix);

    encrypt(text, keyMatrix, encrypted);
    printf("Encrypted: %s\n", encrypted);

    decrypt(encrypted, keyMatrix, decrypted);
    printf("Decrypted: %s\n", decrypted);

    return 0;
}
```

Output:

```

→ cryptography git:(main) × ./hill
Enter plaintext: my cat is sleeping
Key (9 chars, default GYBNQKURP): menuitem
Key Matrix:
12  4 13
20  8 19
 4 12  0

Encrypted: :4YLLIAGQBF6ZHMS0G
Decrypted: UAIYAMWAUSAYKACSAC_

```

LAB 5: Implementing Vigenere Cipher**Objective**

To encrypt and decrypt the given word

Tools Used

VS-Code, gcc compiler

Algorithm: Vigenère Cipher**1) Start****2) Input:**

- Read plaintext message
- Read key phrase

3) Prepare Text:

- Remove non-letters
- Convert text to uppercase

4) Generate Key:

- Repeat key phrase until it matches length of text

5) Encryption:

- For each character:
 - Convert letters to numbers (A=0 to Z=25)
 - Apply: $(\text{Text} + \text{Key}) \bmod 26$
 - Convert result back to letter
- Store encrypted text

6) Display Encrypted Text

7) Decryption:

- Generate same key
- For each character:
 - Apply: $(\text{Text} - \text{Key} + 26) \bmod 26$
 - Convert back to letter

8) Display Decrypted Text

9) End

Source code of Vigenere Cipher

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void prepare_text(char *text) {
    int j = 0;
    for (int i = 0; text[i]; i++) {
        if (isalpha(text[i])) {
            text[j++] = toupper(text[i]);
        }
    }
    text[j] = '\0';
}

void generate_key(char *text, char *key_phrase, char *key) {
    strcpy(key, key_phrase);
    int text_len = strlen(text);
    int key_len = strlen(key_phrase);

    for (int i = 0; i < text_len; i++) {
        if (strlen(key) < text_len) {
            key[i] = key_phrase[i % key_len];
        }
    }
}
```

```

    }
}
key[text_len] = '\0';
}

void encrypt(char *text, char *key_phrase) {
    prepare_text(text);
    char key[200];
    generate_key(text, key_phrase, key);

    for (int i = 0; text[i]; i++) {
        text[i] = ((text[i] - 'A') + (key[i] - 'A')) % 26 + 'A';
    }
}

void decrypt(char *text, char *key_phrase) {
    prepare_text(text);
    char key[200];
    generate_key(text, key_phrase, key);

    for (int i = 0; text[i]; i++) {
        text[i] = ((text[i] - 'A' - (key[i] - 'A') + 26) % 26) + 'A';
    }
}

void print_key(char *text, char *key_phrase) {
    char key[200];
    prepare_text(text);
    generate_key(text, key_phrase, key);
    printf("Generated Key: %s\n", key);
}

int main() {
    char text[200];
    char key_phrase[50];

    printf("Enter text: ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, "\n")] = 0;

    printf("Enter key phrase: ");
    scanf("%s", key_phrase);

    printf("\nOriginal: %s\n", text);

    // Encrypt
    char encrypted[200];
    strcpy(encrypted, text);
    encrypt(encrypted, key_phrase);
    printf("Encrypted: %s\n", encrypted);

    // Show generated key
    print_key(encrypted, key_phrase);

    // Decrypt
    char decrypted[200];
    strcpy(decrypted, encrypted);
    decrypt(decrypted, key_phrase);
    printf("Decrypted: %s\n", decrypted);
}

```



```
    return 0;
}
```

Output:

```
→ cryptography git:(main) x ./vigenere
Enter text: play music
Enter key phrase: theweekend

Original: play music
Encrypted: OYKAWEISV
Generated Key: theweeken
Decrypted: 6LA?M;9IC
```

LAB 6: Implementing One-Time Pad (OTP) Cipher

Objective

To encrypt and decrypt the given word

Tools Used

VS-Code, gcc compiler

Algorithm: One-Time Pad (OTP) Cipher

1. Start

2. Input:

- Read plaintext message from user

3. Initialization:

- Determine length of the message
- Initialize random number generator

4. Generate Key:

- For each character in the message:

- Generate a random number between 0 and 255
- Store it in the key array

5. Encryption:

- For each character i in the message:
 - Convert message character to its ASCII value
 - Apply XOR operation:

$$\text{Cipher}[i] = \text{Message}[i] \oplus \text{Key}[i]$$
 - Store the encrypted byte in cipher array

6. Display:

- Display ASCII values of the input message
- Display generated random key
- Display encrypted (cipher) values

7. Decryption:

- For each character i in cipher:
 - Apply XOR again with the same key:

$$\text{Decrypted}[i] = \text{Cipher}[i] \oplus \text{Key}[i]$$
 - Convert decrypted bytes back to characters

8. Output Decrypted Message

9. End

Source Code OTP Cipher

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

int main() {
    char message[200];
    unsigned char key[200], cipher[200], decrypted[200];

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);
    message[strlen(message)] = 0;

    int len = strlen(message);
```

```
srand(time(NULL)); // Initialize random generator
```

```
// Generate key and encrypt
```

```
for(int i = 0; i < len; i++) {  
    key[i] = rand() % 256;          // random byte (0–255)  
    cipher[i] = message[i] ^ key[i]; // encryption  
}
```

```
// Display input (ASCII)
```

```
printf("\nUser input (ASCII): ");  
for(int i = 0; i < len; i++)  
    printf("%d ", message[i]);
```

```
// Display key
```

```
printf("\nRandom key: ");  
for(int i = 0; i < len; i++)  
    printf("%d ", key[i]);
```

```
// Display encrypted
```

```
printf("\nEncrypted: ");  
for(int i = 0; i < len; i++)  
    printf("%d ", cipher[i]);
```

```
// Decryption
```

```
for(int i = 0; i < len; i++) {  
    decrypted[i] = cipher[i] ^ key[i];  
}  
decrypted[len] = '\0';
```

```
// Display decrypted text
```

```
printf("\nDecrypted: %s\n", decrypted);
```

```
printf("\nWARNING: OTP key must be random and used only once!\n");
```

```
return 0;
```

}

Output of OTP Cipher

```
→ cryptography git:(main) × ./OT
Enter message: virat

User input (ASCII): 118 105 114 97 116
Random key: 185 28 233 140 153
Encrypted: 207 117 155 237 237
Decrypted: virat

WARNING: OTP key must be random and used only once!
```

LAB 7: Implementing Rail Fence Cipher

Objective

To encrypt and decrypt the given text using Rail Fence Cipher

Tools Used

VS-Code, gcc compiler

Algorithm: Rail Fence Cipher

1. Start

2. Input:

- Read plaintext message from user
- Read number of rails

3. Preprocessing:

- Remove non-alphabetic characters
- Convert all letters to uppercase

4. Initialization:

- Create a 2D matrix (rail matrix) initialized with empty values

- Set initial row = 0 and direction = down

5. Encryption:

- For each character in the plaintext:
 - Place the character in the current row and column
 - Move to the next row based on direction:
 - If at top rail → move down
 - If at bottom rail → move up
 - Continue zigzag pattern across rails
- After filling:
 - Read the matrix row by row to generate the ciphertext

6. Display Encrypted Text

7. Decryption:

- Create a similar rail matrix
- Mark zigzag positions using a placeholder (*)
- Fill the marked positions with ciphertext characters row-wise
- Traverse again in zigzag manner to reconstruct original text

8. Output Decrypted Text

Source code of Rail-Fence

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void encrypt_rail_fence(char *text, int rails) {
    int len = strlen(text);
    char rail[100][100] = {0}; // Rail matrix
    int row = 0, dir = 1;      // Current row and direction (1=down, -1=up)

    // Fill rails in zigzag pattern
    for (int i = 0; i < len; i++) {
        rail[row][i] = text[i];

        if (row == 0) dir = 1;      // Top rail - go down
        else if (row == rails - 1) dir = -1; // Bottom rail - go up

        row += dir;
    }

    // Read rails row by row
    int k = 0;
    for (int i = 0; i < rails; i++) {
        for (int j = 0; j < len; j++) {
            if (rail[i][j]) {
```

```

        text[k++] = rail[i][j];
    }
}
text[k] = '\0';
}

```

```

void decrypt_rail_fence(char *cipher, int rails) {
    int len = strlen(cipher);
    char rail[100][100] = {0};
    int row = 0, dir = 1;

```

```

    // Mark positions in zigzag pattern
    for (int i = 0; i < len; i++) {
        rail[row][i] = '*';

```

```

        if (row == 0) dir = 1;
        else if (row == rails - 1) dir = -1;

```

```

        row += dir;
    }

```

```

    // Fill marked positions with cipher text

```

```

    int k = 0;
    for (int i = 0; i < rails; i++) {
        for (int j = 0; j < len; j++) {
            if (rail[i][j] == '*' && k < len) {
                rail[i][j] = cipher[k++];
            }
        }
    }
}

```

```

    // Read in zigzag order

```

```

    row = 0; dir = 1;
    int idx = 0;
    for (int i = 0; i < len; i++) {
        cipher[idx++] = rail[row][i];

        if (row == 0) dir = 1;
        else if (row == rails - 1) dir = -1;

```

```

        row += dir;
    }

```

```

    cipher[idx] = '\0';
}

```

```

void prepare_text(char *text) {
    int j = 0;
    for (int i = 0; text[i]; i++) {
        if (isalpha(text[i])) {
            text[j++] = toupper(text[i]);
        }
    }
}

```

```

    }
    text[j] = '\0';
}

int main() {
    char text[200];
    int rails;

    printf("Enter text: ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, "\n")] = 0;

    printf("Enter number of rails (2-10): ");
    scanf("%d", &rails);

    prepare_text(text);
    printf("Original: %s\n", text);

    // Encrypt
    char encrypted[200];
    strcpy(encrypted, text);
    encrypt_rail_fence(encrypted, rails);
    printf("Encrypted: %s\n", encrypted);

    // Decrypt
    char decrypted[200];
    strcpy(decrypted, encrypted);
    decrypt_rail_fence(decrypted, rails);
    printf("Decrypted: %s\n", decrypted);

    return 0;
}

```

Output

```

→ cryptography git:(main) × ./railfence
Enter text: mobwatch
Enter number of rails (2-10): 2
Original: MOBWATCH
Encrypted: MBACOWTH
Decrypted: MOBWATCH

```

LAB 8: Implementing Euclidean Algorithm and Extended Euclidean Algorithm

Objective

To find the Greatest Common Divisor (GCD) of two numbers using:

- Euclidean Algorithm
- Extended Euclidean Algorithm (to find coefficients)

Tools Used

VS-Code, gcc compiler

Algorithm: Euclidean Algorithm

1. Start

2. Input:

- Read two integers a and b

3. Process:

- Repeat until $b = 0$:
 - Compute remainder: $r = a \% b$
 - Update values:
 - $a = b$
 - $b = r$

4. Output:

- The value of a is the **GCD**

5. End

Algorithm: Extended Euclidean Algorithm

1. Start

2. Input:

- Read two integers a and b

3. Initialization:

- Set:
 - $x1 = 1, y1 = 0$
 - $x2 = 0, y2 = 1$

4. Process:

- While $b \neq 0$:
 - Compute quotient: $q = a / b$
 - Compute remainder: $r = a \% b$
 - Update:
 - $a = b, b = r$
 - Update coefficients:
 - $x = x1 - q * x2$
 - $y = y1 - q * y2$
 - Set:
 - $x1 = x2, x2 = x$
 - $y1 = y2, y2 = y$

5. Output:

- $GCD = a$
- Coefficients x and y such that:
 $ax + by = gcd(a, b)$

6. End

Source code implementing Euclidean and Extended Euclidean Algorithm

```
#include <stdio.h>
#include <stdlib.h>

// Iterative Euclidean Algorithm for GCD
int euclidean_gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Iterative Extended Euclidean Algorithm
// Returns GCD and sets x, y such that a*x + b*y = gcd
int extended_euclidean(int a, int b, int *x, int *y) {
    int x1 = 0, y1 = 1, x2 = 1, y2 = 0;
    if (b == 0) {
        *x = 1;
        *y = 0;
        return a;
    }
    while (b != 0) {
        int q = a / b;
        int temp = b;
        b = a % b;
```

```

    a = temp;
    temp = x1;
    x1 = x2 - q * x1;
    x2 = temp;
    temp = y1;
    y1 = y2 - q * y1;
    y2 = temp;
}
*x = x2;
*y = y2;
return a;
}

int main() {
    int choice, a, b, g, x, y;

    printf("==== Euclidean and Extended Euclidean Algorithms ==== \n");
    printf("1. Basic Euclidean Algorithm (GCD) \n");
    printf("2. Extended Euclidean Algorithm (GCD + Coefficients) \n");
    printf("3. Exit \n");
    printf("Choose option: ");

    while (scanf("%d", &choice) == 1) {
        if (choice == 3) {
            printf("Exiting... \n");
            break;
        }

        if (choice != 1 && choice != 2) {
            printf("Invalid choice! Enter 1, 2, or 3: ");
            continue;
        }

        printf("Enter two positive integers a and b: ");
        if (scanf("%d %d", &a, &b) != 2 || a <= 0 || b <= 0) {
            printf("Please enter valid positive integers! \n");
            continue;
        }

        if (choice == 1) {
            g = euclidean_gcd(a, b);
            printf("GCD(%d, %d) = %d \n \n", a, b, g);
        } else {
            g = extended_euclidean(a, b, &x, &y);
            printf("GCD(%d, %d) = %d \n", a, b, g);
            printf("%d * %d + %d * %d = %d \n \n", a, x, b, y, g);
        }

        printf("Choose option (1/2/3): ");
    }

    return 0;
}

```

Output

```
→ cryptography git:(main) × ./EANDDEEA
=== Euclidean and Extended Euclidean Algorithms ===
1. Basic Euclidean Algorithm (GCD)
2. Extended Euclidean Algorithm (GCD + Coefficients)
3. Exit
Choose option: 1
Enter two positive integers a and b: 2 6
GCD(2, 6) = 2

Choose option (1/2/3): 2
Enter two positive integers a and b: 6 143
GCD(6, 143) = 1
6 * 24 + 143 * -1 = 1

Choose option (1/2/3): 3
Exiting...
```

LAB 9: Diffie-Hellman Key Exchange

Objective

To implement the Diffie-Hellman Key Exchange algorithm for securely generating a shared secret key over an insecure channel.

Tools Used

VS-Code, gcc compiler

Algorithm: Diffie-Hellman Key Exchange

1. Start

2. Input Public Parameters:

- Choose a prime number p
- Choose a primitive root g modulo p

3. Alice's Process:

- Select a private key a
- Compute public key:
 $A = g^a \bmod p$
- Send A to Bob

4. Bob's Process:

- Select a private key b
- Compute public key:
 $B = g^b \bmod p$
- Send B to Alice

5. Key Exchange:

- Alice receives B
- Bob receives A

6. Shared Secret Computation:

- Alice computes:
 $K_1 = B^a \bmod p$
- Bob computes:
 $K_2 = A^b \bmod p$

7. Verification:

- If $K_1 = K_2$, then shared secret key is successfully generated

8. Output:

- Display public keys (A and B)
- Display shared secret key

9. End

Source Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Function for modular exponentiation: (base^exp) mod mod
```

```

// Uses binary exponentiation ( $O(\log \text{exp})$  time) to avoid overflow issues
long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;          // Reduce base first
    while (exp > 0) {
        if (exp % 2 == 1) {      // If exponent is odd
            result = (result * base) % mod;
        }
        base = (base * base) % mod; // Square the base
        exp /= 2;                // Halve the exponent
    }
    return result;
}

int main() {
    long long p, g;              // Public parameters
    long long a, b;              // Private keys
    long long A, B;              // Public keys
    long long K_alice, K_bob;    // Shared secret keys

    printf("=== Diffie-Hellman Key Exchange Demo (BSc CSIT Cryptography Lab) ===\n\n");

    // Input public parameters (common example: p=23, g=5)
    printf("Enter a prime number p: ");
    scanf("%lld", &p);
    printf("Enter primitive root g (mod p): ");
    scanf("%lld", &g);

    // Alice's side
    printf("\nAlice: Enter your private key 'a' ( $1 < a < p-1$ ): ");
    scanf("%lld", &a);
    A = mod_pow(g, a, p);
    printf("Alice computes public key  $A = g^a \bmod p = %lld$ \n", A);

    // Bob's side
    printf("\nBob: Enter your private key 'b' ( $1 < b < p-1$ ): ");
    scanf("%lld", &b);
    B = mod_pow(g, b, p);
    printf("Bob computes public key  $B = g^b \bmod p = %lld$ \n", B);

    // Key exchange simulation
    printf("\n--- Alice and Bob exchange public keys A and B over insecure channel ---\n");

    // Alice computes shared key
    K_alice = mod_pow(B, a, p);
    printf("Alice computes shared secret  $K = B^a \bmod p = %lld$ \n", K_alice);

    // Bob computes shared key
    K_bob = mod_pow(A, b, p);
    printf("Bob computes shared secret  $K = A^b \bmod p = %lld$ \n", K_bob);

    // Verification
    if (K_alice == K_bob) {

```

```

    printf("\nSUCCESS: Shared secret keys match! K = %lld\n", K_alice);
    printf("They can now use this key for symmetric encryption (e.g., AES).\n");
} else {
    printf("\nERROR: Keys do not match (bug in implementation).\n");
}

printf("\n=== Lab Report Usage Tips ===\n");
printf("1. Compile: gcc diffie_hellman.c -o dh\n");
printf("2. Run: ./dh\n");
printf("3. Test with standard values:\n");
printf("  p = 23, g = 5, a = 6, b = 15 → K = 2\n");
printf("  p = 17, g = 3, a = 5, b = 7 → K = 6\n");
printf("4. For report: Include this code, output screenshot, and explain each step.\n");

return 0;
}

```

Output

```

→ cryptography git:(main) × ./dhkeyexchange
=== Diffie-Hellman Key Exchange Demo (BSc CSIT Cryptography Lab) ===

Enter a prime number p: 23
Enter primitive root g (mod p): 5

Alice: Enter your private key 'a' (1 < a < p-1): 6
Alice computes public key A = g^a mod p = 8
Bob: Enter your private key 'b' (1 < b < p-1): 15
Bob computes public key B = g^b mod p = 19

--- Alice and Bob exchange public keys A and B over insecure channel ---
Alice computes shared secret K = B^a mod p = 2
Bob computes shared secret K = A^b mod p = 2

SUCCESS: Shared secret keys match! K = 2
They can now use this key for symmetric encryption (e.g., AES).

```

LAB 10: RSA Algorithm

Objective

To implement the RSA algorithm for encryption and decryption of a message using public and private keys.

Tools Used

VS-Code, gcc compiler

Algorithm: RSA Encryption and Decryption

1. Start

2. Input:

- Choose two prime numbers p and q

3. Compute:

- Calculate:
 - $n = p \times q$
 - $\phi(n) = (p - 1)(q - 1)$

4. Choose Public Key:

- Select an integer e such that:
 - $1 < e < \phi(n)$
 - $\gcd(e, \phi(n)) = 1$

5. Compute Private Key:

- Find d such that:
 - $(d \times e) \bmod \phi(n) = 1$

6. Encryption:

- Convert message M into numeric form
- Compute ciphertext:
 - $C = M^e \bmod n$

7. Decryption:

- Compute original message:
 - $M = C^d \bmod n$

8. Output:

- Public key: (e, n)
- Private key: (d, n)
- Ciphertext and decrypted message

Source Code RSA

```
#include <stdio.h>
```

```
// Function to compute (base^exp) % mod
```

```
long long mod_pow(long long base, long long exp, long long mod) {
```

```
    long long result = 1;
```

```
    base = base % mod;
```

```

while (exp > 0) {
    if (exp % 2 == 1)
        result = (result * base) % mod;

    base = (base * base) % mod;
    exp = exp / 2;
}
return result;
}

```

// Function to compute gcd

```

long long gcd(long long a, long long b) {
    while (b != 0) {
        long long temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

```

// Function to find modular inverse (d)

```

long long mod_inverse(long long e, long long phi) {
    for (long long d = 1; d < phi; d++) {
        if ((d * e) % phi == 1)
            return d;
    }
    return -1;
}

```

```

int main() {
    long long p, q, n, phi;
    long long e, d;
    long long message, encrypted, decrypted;
}

```



```
printf("Enter first prime number (p): ");
```

```
scanf("%lld", &p);
```

```
printf("Enter second prime number (q): ");
```

```
scanf("%lld", &q);
```

```
// Step 1: Compute n and phi
```

```
n = p * q;
```

```
phi = (p - 1) * (q - 1);
```

```
// Step 2: Choose e
```

```
printf("Enter public key exponent (e): ");
```

```
scanf("%lld", &e);
```

```
// Ensure gcd(e, phi) = 1
```

```
if (gcd(e, phi) != 1) {
```

```
    printf("Invalid e. It must be coprime with  $\phi(n)$ .\n");
```

```
    return 1;
```

```
}
```

```
// Step 3: Compute d
```

```
d = mod_inverse(e, phi);
```

```
if (d == -1) {
```

```
    printf("Modular inverse not found.\n");
```

```
    return 1;
```

```
}
```

```
printf("\nPublic Key: (e = %lld, n = %lld)\n", e, n);
```

```
printf("Private Key: (d = %lld, n = %lld)\n", d, n);
```

```
// Input message
```

```
printf("\nEnter numeric message (M < n): ");
```

```
scanf("%lld", &message);
```

```

// Encryption
encrypted = mod_pow(message, e, n);
printf("Encrypted message: %lld\n", encrypted);

// Decryption
decrypted = mod_pow(encrypted, d, n);
printf("Decrypted message: %lld\n", decrypted);

return 0;
}

```

Output

```

→ cryptography git:(main) × ./RSA
Enter first prime number (p): 3
Enter second prime number (q): 11
Enter public key exponent (e): 7

Public Key: (e = 7, n = 33)
Private Key: (d = 3, n = 33)

Enter numeric message (M < n): 5
Encrypted message: 14
Decrypted message: 5

```